ChatGPT

FinStream AI – Real-Time Transaction Intelligence & Anomaly Detection

Executive Summary: FinStream AI is a cutting-edge platform designed to ingest and analyze **streaming financial transactions** in real time to detect anomalies and fraud. It uses **Apache Kafka** for event ingestion and **Spark Structured Streaming** (or Python stream consumers) for processing, along with machine learning for risk scoring. Live KPIs and alerts are displayed on a dashboard and can trigger notifications. In today's financial sector, real-time monitoring is critical: researchers note that detecting anomalies in high-volume payment streams is "like finding a needle in a haystack"【32†L575-L584】. By combining high-throughput event streaming【27†L75-L84】, scalable stream processing【38†L58-L66】, and explainable ML models, FinStream AI delivers actionable insights within seconds.

FinStream AI's architecture is end-to-end: transaction events (e.g. point-of-sale payments, online transfers) are published to Kafka topics, where a stream processor (Spark or Python) ingests them, computes features (e.g. transaction velocity, geolocation, merchant profile), and feeds them to ML models for scoring. Scores and features are stored in a fast feature store (Redis) and a database (PostgreSQL). A backend API serves this data to a live dashboard, showing metrics (volumes, fraud rates) and flagged transactions, and issues alerts for suspicious activity. The solution emphasizes scalability (thousands of tx/sec), low latency (sub-second processing), and transparency (using SHAP for feature explanations). Recent industry research similarly advocates layered real-time ML frameworks for payment monitoring【32†L589-L598】, and FinStream AI adopts best practices to build an **industry-quality, production-like prototype** for a B.Tech capstone.

Key Objectives and Scope

- **Real-time Ingestion and Analytics:** Continuously capture financial transaction events via Kafka; process them with Spark (or equivalent) with end-to-end latency of ≈100–500 ms.
- **Anomaly/Fraud Detection:** Apply ML models to score each transaction for fraud risk. Use a layered approach: first classify "typical" vs "unusual" (supervised) and then apply unsupervised anomaly detection on unusual cases【32†L575-L584】【32†L589-L598】.
- **Explainability:** Use SHAP or similar to attribute model decisions to features, aiding interpretation and trust.
- **Dashboard & Alerts:** Provide a live BI dashboard (Grafana/React) with key metrics (e.g. txn/sec, fraud alerts, average score) and highlight suspicious transactions. Generate alerts (e.g. email, logs) for flagged events.
- **Scalability & Reliability:** Architect with Kafka for fault-tolerant storage and horizontal scaling; use Spark or Python for parallel stream processing; target a throughput of thousands of events/sec with built-in recovery semantics.
- **Data & Implementation:** Utilize public or synthetic datasets (e.g. Kaggle PaySim mobile-money data【13†L193-L202】, IBM/TabFormer's synthetic credit-card data【22†L241-L249】) to simulate live streams. Cover all components from data producer to user-facing dashboard.

Architecture Diagram

```
flowchart LR
    subgraph DataSources
      A[Branch/ATM/Online API] -->|Transaction JSON| Kafka
      B[Mobile App / POS] -->|Transaction JSON| Kafka
    end
    Kafka[Apache Kafka<br/>(topics: raw_txn)] -->|stream| StreamProcessor
    StreamProcessor[Stream Processor<br/>(Spark Structured Streaming or Python)]
    StreamProcessor -->|Features, Scores| Redis[(Redis Cache)]
    StreamProcessor -->|Scored Events| Postgres[(PostgreSQL DB)]
    StreamProcessor -->|Alert| AlertService{Anomaly Detection}
    Redis --> BackendAPI
    Postgres --> BackendAPI[Backend API<br/>(FastAPI)]
    BackendAPI --> Dashboard[Web Dashboard<br/>(React/Grafana)]
    AlertService -->|Notifications| Dashboard
```

Fig: High-level architecture of FinStream AI. Live transaction events flow into Kafka, are processed by a streaming engine (Spark or Python), stored and scored, then exposed via an API to a dashboard and alerting system.

Core Components and Responsibilities

Component	Technology	Function (Responsibilities)
Event Producer	Python Kafka lib	Simulate/generate transaction events or read from CSV and publish JSON to Kafka topics.
Kafka Cluster	Apache Kafka	High-throughput message broker storing incoming transaction streams (raw_txn topic).
Stream Processor	Spark Structured Streaming (or Python Kafka consumer)	Consume Kafka events, perform data cleaning & aggregation, join with reference data, compute features, and produce scored events.
Feature Store (Cache)	Redis	Store real-time aggregated features (e.g. sliding-window counts, customer profiles) for low-latency lookup.
Database	PostgreSQL	Persist raw transactions, feature snapshots, and prediction scores for historical queries and dashboards.
ML Model (Scoring)	Scikit-learn / XGBoost / Autoencoder	Pre-trained models (classification/anomaly detection) to compute fraud-risk scores on each event.

Component	Technology	Function (Responsibilities)
Explainability	SHAP (Python)	Compute SHAP values for model predictions to identify top contributing features for any flagged transaction.
Backend API	Python (FastAPI)	REST API to serve real-time scores and metrics from Postgres/Redis to the dashboard or external clients.
Dashboard & Alert	React/Flask/Grafana	Web UI displaying live metrics (TPS, fraud rate, alerts). Generates visual alerts/notifications on anomalies.

Data Sources and Schemas

We use open/synthetic transaction data to simulate live streams. Examples include the **PaySim** mobile-money simulator dataset【13†L193-L202】 and the synthetic Kaggle credit-card dataset【22†L241-L249】. Each transaction event might contain fields like:

Field	Type	Description
`transaction_id`	string	Unique ID (UUID)
`timestamp`	string	ISO8601 UTC time of transaction
`account_id`	string	Customer or account identifier
`amount`	float	Transaction amount (e.g. 123.45)
`currency`	string	Currency code (e.g. USD)
`merchant`	string	Merchant name or ID
`category`	string	Transaction category (ecommerce, food, etc.)
`city`	string	City or location
`device_id`	string	Device or terminal ID
`channel`	string	Channel (POS, online, mobile, ATM)
`card_present`	bool	Was card physically present (yes/no)
`is_fraud`	int	(Optional) Ground-truth fraud label (0/1).

Sample JSON event:

```
{
  "transaction_id": "tx1000001",
  "timestamp": "2026-09-30T12:34:56Z",
  "account_id": "acct_501",
```

```
  "amount": 1500.75,
  "currency": "USD",
  "merchant": "Amazon",
  "category": "ecommerce",
  "city": "Seattle",
  "device_id": "dev_09",
  "channel": "online",
  "card_present": false,
  "is_fraud": 0
}
```

Transactions can be streamed by reading rows from CSVs or generated via synthetic generators (e.g. **PaySim**【13†L193-L202】) and publishing them to Kafka at a controlled rate. Historical logs and reference tables (merchant risk ratings, customer profiles) can be stored in Postgres for enrichment. A Redis instance may hold fast lookup tables or computed aggregates (e.g. count of transactions per account in last hour) that streaming jobs update in real time.

Machine Learning Approach

- **Algorithms:** We consider a variety of models. For supervised fraud classification: **XGBoost** or **Random Forest** on engineered features. For anomaly detection: **Isolation Forest**, **Autoencoders**, or LSTM-based methods on sequences. We implement a **layered model**: first a classifier to filter "normal" transactions, then an unsupervised model to score the remainder for anomalies (per 【32†L575-L584】【32†L589-L598】).
- **Feature Engineering:** Numeric features (amount, time since last txn), categorical (merchant, city via embeddings), and aggregated statistics (velocity features like txns per minute, total spend in last 24h). Additional features: distance between successive locations, typical spending profiles, card-presence flag, device usage patterns, etc. Use of sliding-window or bloom filters for fast computation.
- **Training vs Online:** Models are initially trained offline on labeled data (synthetic or historical) with techniques like grid-search and cross-validation. In production, we periodically re-train (weekly/monthly) on recent data. For online adaptation, light models (e.g. incremental classifiers or online random forests) or retraining can be triggered if concept drift is detected.
- **Evaluation:** We use **precision/recall/F1** and **ROC-AUC** on holdout data to benchmark models. Since fraud events are rare, PR curves and false-positive rate control are important. We also measure system metrics (latency, throughput). For each alert, SHAP values provide explanations: e.g. "transaction flagged as high-risk due to unusually high amount and rare merchant category." Explainability aids debugging and trust.

Real-Time Requirements

- **Latency Target:** End-to-end latency from event ingress to alert should be on the order of hundreds of milliseconds (<1 s). For example, the pipeline in a similar proof-of-concept achieved ≈30 ms per transaction【26†L169-L177】 (but realistic SLA ~100–500ms).
- **Throughput:** Design for at least 500–1000 transactions per second (tps) to handle bursts. Kafka partitioning and Spark parallelism will be used to scale. (Kafka is proven to handle millions of events/

sec【27†L114-L123】; Spark streaming can process large volumes with micro-batching【38†L58-L66】.)

- **Scaling:** Kafka topic (raw_txn) should have multiple partitions. The stream processor runs in a cluster mode or multi-threaded Python, with enough resources to meet throughput. Redis and Postgres can be clustered/sharded if needed.
- **Reliability:** Use Kafka retention and replication for durability. Ensure exactly-once (or at-least-once + idempotency) processing semantics in Spark to avoid misses/duplicates. Checkpointing of state (in Spark) or external state store (Redis) will ensure recovery from failures.

Demo Plan and Scenarios

During the viva/demo, we will simulate realistic scenarios to showcase the system:

- **Normal Load:** Stream routine transactions (non-fraud) and show dashboard metrics updating (throughput, volume by category, etc.). Verify that no alerts are triggered.
- **Fraud Spike:** Inject a burst of fraudulent-looking transactions (e.g. many high-value TXNs, new merchants, cross-border activity). The system should flag these in real time. Show the alert log and mark anomalies on the dashboard.
- **Feature/Explainability:** Click on a flagged transaction in the UI to display SHAP-based feature contributions (e.g. “high amount”, “unusual merchant”). Demonstrate understanding of why the ML model scored it.
- **What-if Dashboard:** (Optional) If implemented, show how changing a feature (e.g. lowering transaction amount) would affect the risk score.
- **Fault Tolerance:** Simulate a component failure (e.g. restart the stream processor) and show that Kafka provides buffered data and the system recovers without data loss.
- **Throughput Test:** Gradually increase event rate and observe system throughput and latency metrics (logs). Ensure no bottleneck up to target load.

The demo will highlight the full data flow: pushing events via a Python producer, Kafka topics, real-time processing with Spark, score output into Redis/DB, and live dashboard update. Sample log snippets will show alerting with timestamps.

Implementation Roadmap

Milestone	Tasks	Duration (weeks)
1. Setup & Data Prep	Setup Kafka, Spark, Redis, Postgres locally (Docker). Collect/prepare datasets (PaySim, Kaggle). Define data schema and Kafka topics.	1–2
2. Producer & Kafka	Implement Python Kafka producer to publish synthetic transactions at configurable rate. Test Kafka topics.	1
3. Stream Processing	Develop Spark Structured Streaming (or Kafka-Python) job: consume raw_txn topic, parse JSON, basic cleansing, window aggregations.	2

Milestone	Tasks	Duration (weeks)
4. Feature Engineering	Extend processor to compute features (velocity, merchant risk, geo features). Store features in Redis and Postgres.	1–2
5. ML Modeling	Train initial models offline (XGBoost classifier + Isolation Forest). Integrate model loading into the stream job to score each event.	2
6. Explainability	Add SHAP integration to compute feature contributions on flagged events. Store explanations in DB.	1
7. Backend & API	Build FastAPI service to query Postgres/Redis for metrics and alerts. Provide endpoints for dashboard.	1
8. Dashboard/Alerts	Create React or Grafana dashboard connecting to API. Display live charts (txn rate, risk distribution) and list anomalies. Configure alert notifications (email/log).	1–2
9. Testing & Tuning	End-to-end testing, performance tuning (batch sizes, parallelism). Tune ML model thresholds.	1
10. Documentation & Report	Write README, prepare report and slides, polish code on GitHub. Practice demo scenarios.	1

Total Duration: ~10–12 weeks. Tasks can run in parallel (e.g. Dashboard work can begin while models are training).

Security, Privacy, and Ethics

- **Data Privacy:** Transaction data is highly sensitive. In a real deployment, enforce encryption in transit (TLS), restrict access, and possibly anonymize PII (e.g. hash account IDs). Comply with regulations like PCI DSS and GDPR by minimizing data retention and using privacy-preserving techniques (tokenization).
- **Access Control:** Secure Kafka and database with authentication. The backend API should enforce user authentication (JWT/OAuth) if exposed.
- **Bias and Fairness:** While aiming to catch fraud, the model may learn biases (e.g. flagging certain demographics unfairly). We must ensure transparency (SHAP helps) and regularly audit false positives. Avoid using protected attributes (race, gender) as features.
- **Explainability:** Provide human-understandable reasons for alerts so that operators can make informed decisions. Over-reliance on opaque models can have serious ethical implications if false alarms affect users.
- **Security Testing:** Simulate adversarial scenarios (e.g. transaction injection, denial-of-service) to ensure robustness. Do not expose internal endpoints.
- **Responsible Use:** Emphasize that the system is a **decision-support tool** (to flag anomalies for review), not an automated verdict. Disclaimers in the UI can remind analysts to corroborate alerts with additional checks.

README and Resume Snippets

- **README (project description):**
  "FinStream AI is an open-source real-time financial transaction intelligence platform. It ingests streaming transactions via Apache Kafka, processes them with Spark Structured Streaming and Python for feature extraction, and applies ML models (XGBoost/Autoencoder) to score fraud risk. Features and scores are stored in Redis/Postgres and served through a FastAPI backend to a live dashboard built in React/Grafana. The dashboard displays live KPIs (transaction rate, fraud alerts, etc.) and allows investigation of anomalies. The system uses SHAP for explainability and alerts users to suspicious transactions within milliseconds, demonstrating a full end-to-end streaming analytics architecture."

- **Resume (one-line bullet):**
  "Developed FinStream AI, a real-time financial transaction analytics platform (Kafka, Spark, Python, XGBoost) to monitor live payment streams and detect anomalies with low-latency alerts."
